

Laboratório de SOC integrado

Fase 2 — Monitoramento de infraestrutura com Zabbix

Campo	Valor
Autor	Kauan Trinchetti
Fase	2 de 7 — Monitoramento
Componente implantado	Zabbix 7.0.29 LTS (server, frontend e banco em containers)
Ambiente	VM Ubuntu Server 24.04 · 5 GB RAM · Docker Engine
Endpoints monitorados	srv-linux (VM do laboratório) e ws-windows (estação Windows)
Status	Concluída

1. Objetivo

Implantar a camada de monitoramento de disponibilidade e capacidade do laboratório, com coleta ativa em endpoints Linux e Windows, e preparar o ambiente para a integração da Fase 4, na qual eventos de severidade elevada passarão a abrir chamados automaticamente no GLPI.

Como objetivo secundário, estabelecer observabilidade sobre os próprios containers que sustentam o laboratório, de modo que a indisponibilidade de um componente do SIEM seja detectada pela camada de monitoramento.

2. Decisão de arquitetura

O planejamento inicial previa uma instância única de banco de dados atendendo GLPI e Zabbix, com o objetivo de reduzir o consumo de memória. No momento da implantação, o GLPI já se encontrava configurado, com categorias de chamado, conta de serviço e tokens de API emitidos.

A decisão foi reavaliada com base nos seguintes critérios:

Critério	Avaliação
Custo de manter instâncias separadas	Aproximadamente 350 MB de memória adicional
Custo de consolidar	Migração de dados e reemissão de tokens, com risco de perda de configuração
Margem disponível na VM	Aproximadamente 1,9 GB

Optou-se por manter as stacks separadas. Havia folga de memória suficiente e o risco associado à migração superava o ganho previsto.

3. Implantação

3.1 Stack de containers

A stack foi declarada em Docker Compose, contendo banco de dados, servidor e interface web. O serviço de agente previsto no arquivo original foi removido, conforme descrito na seção 3.2.

Durante a primeira inicialização, o servidor permanece em espera enquanto o banco executa sua rotina de criação inicial, e em seguida importa o schema. A importação não deve ser interrompida: um reinício durante esse processo deixa o banco com estrutura criada e sem dados, condição relatada na seção 6.

3.2 Agentes

Optou-se pelo agente nativo em detrimento do agente em container. O agente em container observa o namespace do próprio container, produzindo métricas de sistema que não correspondem ao host — comportamento inadequado para monitoramento de capacidade.

```
Executing: /usr/lib/systemd/systemd-sysv-install enable zabbix-agent2
kauan@lab-server:~$ sudo systemctl status zabbix-agent2
● zabbix-agent2.service - Zabbix Agent 2
   Loaded: loaded (/usr/lib/systemd/system/zabbix-agent2.service; enabled; preset: enabled)
   Active: active (running) since Fri 2026-08-14 12:45:07 UTC; 5min ago
     Main PID: 2497 (zabbix_agent2)
        Tasks: 7 (limit: 5803)
      Memory: 3.8M (peak: 4.3M)
         CPU: 229ms
    CGroup: /system.slice/zabbix-agent2.service
            └─2497 /usr/sbin/zabbix_agent2 -c /etc/zabbix/zabbix_agent2.conf

ago 14 12:45:06 lab-server systemd[1]: Starting zabbix-agent2.service - Zabbix Agent 2...
ago 14 12:45:06 lab-server zabbix_agent2[2492]: Validating configuration file "/etc/zabbix/zabbix_agent2.conf"
ago 14 12:45:07 lab-server zabbix_agent2[2492]: Validation successful
ago 14 12:45:07 lab-server systemd[1]: Started zabbix-agent2.service - Zabbix Agent 2.
ago 14 12:45:07 lab-server zabbix_agent2[2497]: Starting Zabbix Agent 2 (7.0.29)
ago 14 12:45:07 lab-server zabbix_agent2[2497]: Zabbix Agent2 hostname: [Zabbix server]
ago 14 12:45:07 lab-server zabbix_agent2[2497]: Press Ctrl+C to exit.
```

Figura 1 — Serviço zabbix-agent2 em execução na VM

Na estação Windows, o agente foi instalado via MSI, com o endereço do servidor informado nos dois campos: o primeiro autoriza as verificações passivas, o segundo define o destino das verificações ativas.

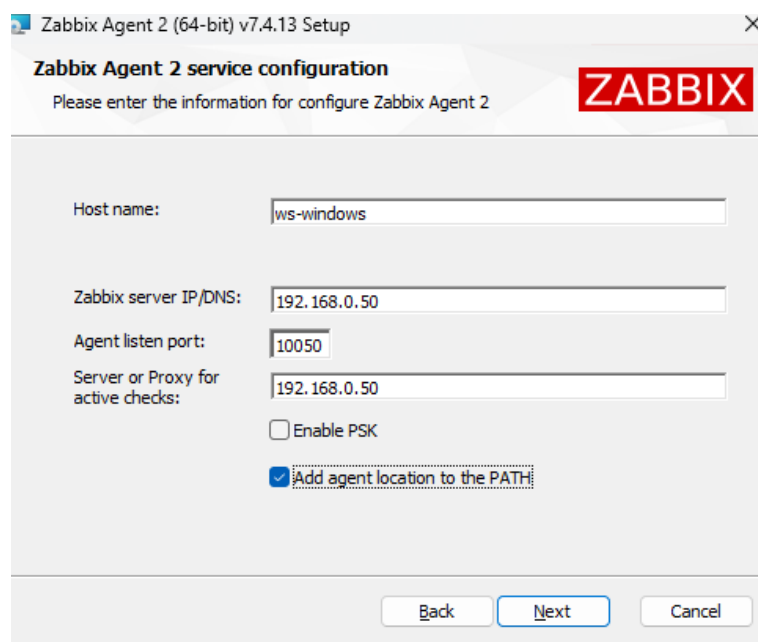


Figura 2 — Configuração do agente na estação Windows

```
PS C:\WINDOWS\system32> Get-Service "Zabbix Agent 2"

Status Name DisplayName
-----
Running Zabbix Agent 2 Zabbix Agent 2
```

Figura 3 — Serviço em execução no Windows

4. Diagnóstico de conectividade

Após o cadastro dos hosts, ambos permaneceram indisponíveis com a mensagem de tempo esgotado na conexão TCP com a porta do agente.

A distinção entre as duas mensagens de erro possíveis orientou o diagnóstico:

Mensagem	Interpretação
Connection refused	O destino respondeu rejeitando a conexão. Serviço parado ou porta fechada.
Timed out	Não houve resposta. O pacote foi descartado no caminho, tipicamente por filtro de pacotes.

A verificação local confirmou que o agente estava ativo e em escuta em todas as interfaces na porta 10050. O firewall do host (ufw) encontrava-se habilitado, autorizando apenas a porta 22.

Um aspecto relevante do ambiente: o servidor Zabbix é executado em container. As conexões que ele estabelece com o agente do host originam-se do endereço da bridge do Docker, e não do endereço da rede local. A regra de firewall precisa contemplar essa origem.

A liberação foi aplicada de forma restrita à faixa de endereços dos containers, e não à rede local:

```
sudo ufw allow from 172.16.0.0/12 to any port 10050 proto tcp
```

A validação foi realizada a partir do container responsável pela coleta, antes de retornar à interface web:

```
docker compose exec zabbix-server zabbix_get -s 192.168.0.50 -k agent.ping
```

```
kauan@lab-server:~/lab-soc/zabbix$ sudo ufw status numbered
status: active

To Action From
--
1] 22/tcp ALLOW IN Anywhere
2] 10050/tcp ALLOW IN 172.16.0.0/12 # Zabbix server container
3] 22/tcp (v6) ALLOW IN Anywhere (v6)

kauan@lab-server:~/lab-soc/zabbix$ cd ~/lab-soc/zabbix && docker compose exec zabbix-server zabbix_get -s 192.168.0.50 -k agent.ping
zabbix_get[1]: connection timed out: No response from 192.168.0.50:10050

kauan@lab-server:~/lab-soc/zabbix$ docker stats --no-stream --format "table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}"
NAME CPU % MEM USAGE / LIMIT
zabbix-web 4.11% 80.43MiB / 4.803GiB
zabbix-server 0.95% 56.02MiB / 4.803GiB
zabbix-mysql 6.51% 666.2MiB / 4.803GiB
lpi 0.02% 178.3MiB / 4.803GiB
lpi-db 0.83% 482.7MiB / 4.803GiB
```

Figura 4 — Regra aplicada, validação do agente e consumo da stack

5. Inventário e observabilidade

5.1 Hosts e templates

Foram cadastrados dois hosts, associados aos templates padrão correspondentes ao sistema operacional. O host de exemplo criado pela instalação foi desabilitado: sua interface aponta para o endereço de loopback, que dentro do container corresponde ao próprio container, gerando um alerta permanente sem ação corretiva possível.

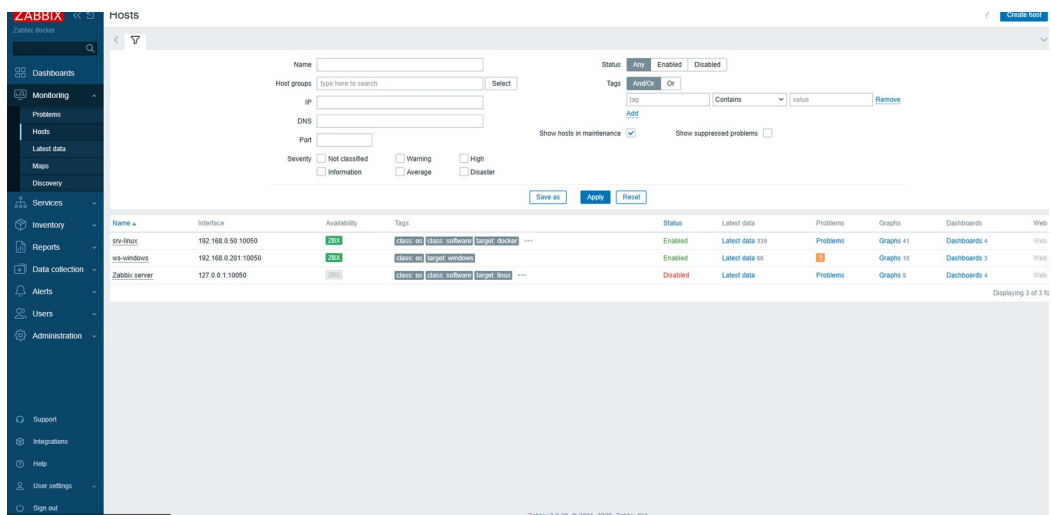


Figura 5 — Hosts cadastrados

5.2 Containers do laboratório

O plugin de Docker do agente foi habilitado sobre o socket do daemon, com o usuário do agente incluído no grupo correspondente. Cada container passou a expor métricas próprias de CPU, memória e estado.

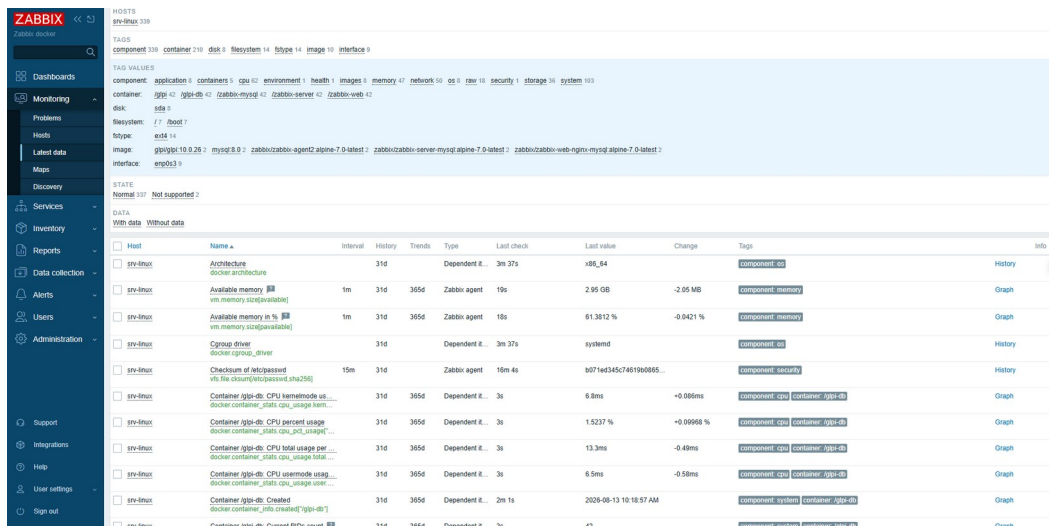


Figura 6 — Métricas por container

Consumo observado com GLPI e Zabbix em operação simultânea:

Container	Memória
zabbix-mysql	666 MB
glpi-db	483 MB
glpi	178 MB
zabbix-web	80 MB
zabbix-server	56 MB
Total	1,43 GB

As instâncias de banco de dados respondem por aproximadamente 80% do consumo da stack, o que quantifica o custo da decisão registrada na seção 2.

6. Trigger e teste controlado

Além das triggers herdadas dos templates, foi criada uma expressão própria sobre o item de ocupação do sistema de arquivos raiz. A chave do item foi obtida na lista de itens do host: no template da versão 7.0 as métricas de sistema de arquivos são itens dependentes de um item mestre, e a chave difere da utilizada em versões anteriores.

```
last(/srv-linux/vfs.fs.dependent.size[/,pused])>40
```

O teste foi conduzido ajustando temporariamente o limiar para um valor próximo à ocupação corrente e elevando a ocupação de forma controlada até cruzá-lo. Após o registro do evento, o arquivo foi removido e o limiar retornado ao valor operacional.

```
kauan@lab-server:~/lab-soc/glpi$ df -h /
Filesystem                Size      Used Avail Use% Mounted on
/dev/mapper/ubuntu--vg-ubuntu--lv  38G   11G   25G   31% /
kauan@lab-server:~/lab-soc/glpi$ sudo falldate -l 5G /root/teste.img
kauan@lab-server:~/lab-soc/glpi$ df -h /
Filesystem                Size      Used Avail Use% Mounted on
/dev/mapper/ubuntu--vg-ubuntu--lv  38G   16G   20G   45% /
kauan@lab-server:~/lab-soc/glpi$ sudo rm /root/teste.img
kauan@lab-server:~/lab-soc/glpi$ df -h /
Filesystem                Size      Used Avail Use% Mounted on
/dev/mapper/ubuntu--vg-ubuntu--lv  38G   11G   25G   31% /
kauan@lab-server:~/lab-soc/glpi$
```

Figura 7 — Elevação e retorno da ocupação do sistema de arquivos

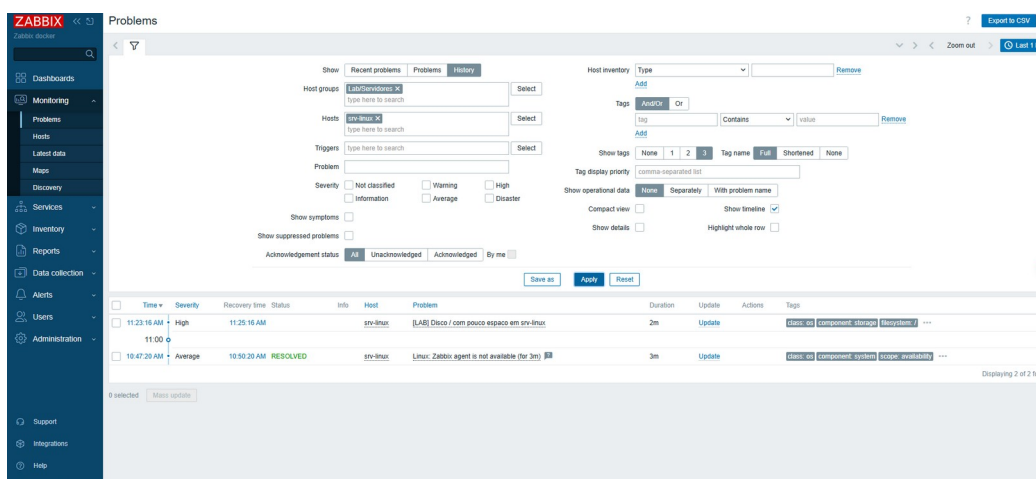


Figura 8 — Eventos registrados e encerrados

7. Ocorrências e resolução

Ocorrência	Causa raiz	Resolução
Diretório do projeto ausente	Os arquivos da Fase 1 haviam sido criados após elevação de privilégio, residindo no diretório pessoal do usuário root	Movido para o diretório do usuário comum, com ajuste de propriedade. Os volumes nomeados são independentes do caminho, sem impacto no serviço
Conflito na porta 10050	O container de agente previsto no compose disputava a porta com o agente nativo já instalado	Remoção do serviço de agente da stack
Frontend inacessível com erro de configuração	Reinício do servidor durante a importação do schema deixou as tabelas criadas sem os dados iniciais	Remoção dos volumes do projeto e nova inicialização sem interrupção

Ocorrência	Causa raiz	Resolução
Hosts indisponíveis por tempo esgotado	Firewall do host autorizava apenas a porta 22; a origem das conexões é a bridge do Docker	Regra restrita à faixa de endereços dos containers
Host Windows aparentemente disponível	A interface apontava para o endereço da VM; o agente Linux respondia às verificações de disponibilidade	Correção do endereço da interface para o da estação
Ausência de espaço em disco	Tentativa de alocação superior ao espaço livre; ver seção 8	Remoção do arquivo e expansão do volume lógico

8. Achados relevantes

8.1 Volume lógico com metade da capacidade alocada

A instalação padrão do Ubuntu Server com LVM alocou aproximadamente 19 GB ao volume lógico, de um disco de 40 GB. A capacidade restante permaneceu livre no grupo de volumes, sem uso e sem indicação na interface do sistema.

A correção foi aplicada com o sistema em operação, sem indisponibilidade:

```
sudo lvextend -l +100%FREE /dev/ubuntu-vg/ubuntu-lv
sudo resize2fs /dev/ubuntu-vg/ubuntu-lv
```

8.2 Ponto cego de coleta

O sistema de arquivos raiz atingiu 100% de ocupação sem que qualquer alerta de disco fosse gerado. A ocupação ocorreu durante a janela em que o agente estava inalcançável pela regra de firewall, de modo que não houve coleta e, consequentemente, nenhuma trigger foi avaliada.

O único evento aberto no período foi o de indisponibilidade do agente, com duração de três minutos, e foi ele que evidenciou a condição.

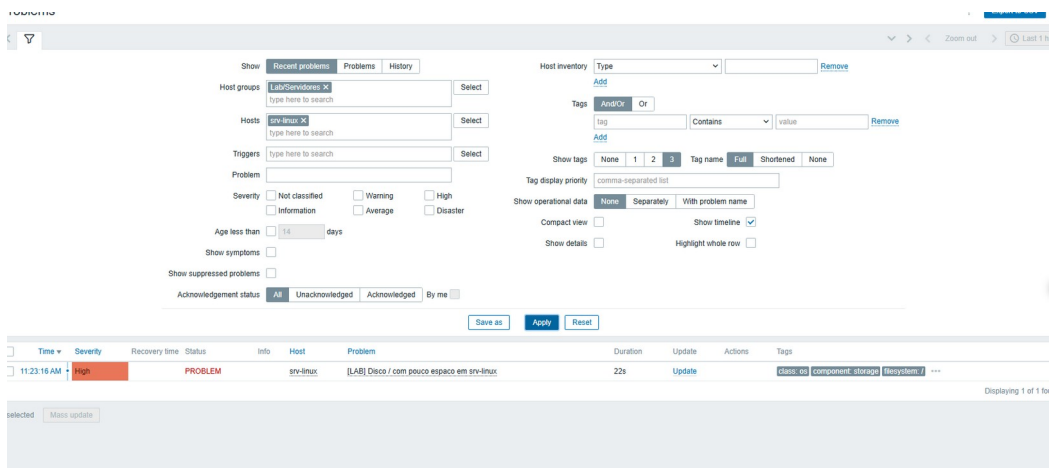


Figura 9 — Evento de indisponibilidade do agente

A observação sustenta a necessidade de triggers baseadas em ausência de dados: um sistema de monitoramento não registra o que não coleta, e a ausência de alertas não deve ser interpretada como ausência de incidentes.

9. Resultado da fase

- Zabbix 7.0.29 LTS em operação, com servidor, frontend e banco em containers
- Dois endpoints monitorados com os templates padrão de sistema operacional
- Agente nativo em ambos os endpoints, com acesso restrito por firewall à origem esperada
- Métricas por container disponíveis, incluindo os componentes do próprio laboratório
- Trigger própria criada e validada por teste controlado de limiar
- Consumo total da stack de 1,43 GB, dentro da margem definida para o ambiente

10. Próximas fases

Fase	Escopo
3	Wazuh 4.14.7: agentes Linux e Windows, monitoramento de integridade de arquivos, regras de correlação e resposta ativa
4	Integração: módulo integrator do Wazuh e media type webhook do Zabbix para a API REST do GLPI
5	Dashboards operacionais nas três ferramentas
6	Simulação de incidentes e validação da detecção